

SOFTWARE ENGINEERING INTERVIEW — QUICK GUIDE

CodeTrackr

A plain-English 20-page summary of the full technical prep document — read once or twice and you can hold your own in an interview

Condensed from `CodeTrackr_Interview_Preparation.md` (64 pp)

Full versions in `docs/interview-preparation/`

REAL = in the code now · **BETTER** = a suggested improvement

Contents

1. What CodeTrackr is (in one breath)
2. Three ways to explain it
3. The big picture (architecture)
4. The VS Code extension
5. The API key (this is the #1 interview topic)
6. The database (MongoDB)
7. The backend and API
8. The frontend (React)
9. Insights — "is this machine learning?"
10. The social features — leaderboard and groups
11. Login and security
12. Scaling up — 10k, 100k, 1M users
13. Testing and deployment
14. Quick Q&A (memorise these)
15. "Defend your project" — the tough follow-ups
16. Final reminders

1. What CodeTrackr is (in one breath)

CodeTrackr watches how you code and shows you the numbers.

There are three parts:

1. **A VS Code extension.** It runs quietly in your editor. It notices what file you're in, what language, how much you type, how many commands you run in the terminal, when you commit — and every 30–90 seconds it sends a small summary to a server.
2. **A backend server** (Node + Express + MongoDB). It checks who the data belongs to using a personal key, saves each summary as one database record, and does the maths when the website asks for it.
3. **A React website** (the dashboard). You log in with Google. It shows charts of your coding, a global leaderboard, study "groups", goal tracking, and an "Insights" page with things like your most productive hour.

Two different logins:

- The **website** uses Google sign-in → the server gives you a JWT stored in a cookie.
- The **extension** uses a **64-character API key** you copy from the website.

Where it runs: website on Vercel, server on Render, database on MongoDB Atlas.

The one honest thing to say first

The "Insights" page is **not machine learning**. It's ordinary maths — averages, medians, a "how steady are you" score. No model, no training, no AI. That was a deliberate choice so every number can be explained. (More in section 9.)

2. Three ways to explain it

30 seconds

"CodeTrackr is a coding-activity tracker. You install a VS Code extension, paste in a key from the website, and it records what you work on — time per language, editor activity, terminal commands, git commits — and sends it to my backend. The React dashboard turns that into charts, a leaderboard, study groups, goals, and an insights page. It's the MERN stack — MongoDB, Express, React, Node — plus a TypeScript VS Code extension."

1 minute

"It fixes a real gap: most developers can't tell you where their coding time went. My VS Code extension hooks into editor, terminal, git and window events and keeps running counters — lines added, 'churn' (code you wrote then deleted), focused minutes, command success rates. Every 30 seconds it sends one activity record to an Express API, checked by a per-user API key. MongoDB stores each record. The React frontend shows daily and weekly Chart.js graphs, a leaderboard ranked by coding hours, groups you can create and join (public or password-protected), goals on a calendar with deadline reminders driven by a cron job, and an insights page that works out five productivity metrics using plain statistics."

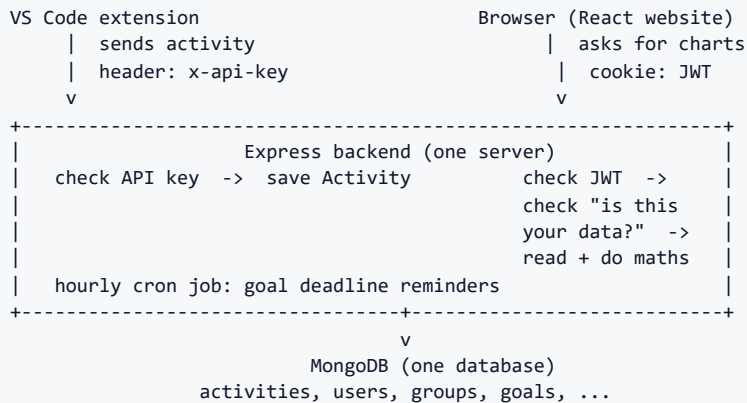
2 minutes

Add this to the 1-minute version:

"The interesting design problem was linking the extension to the account without a full login screen inside VS Code. I used an API-key model: on first sign-in the server generates a random 64-character key and stores it on the user. The extension sends it as a header; the server looks up the user by that key. It works, but in a review I'd call it what it actually is — a password-like credential that never expires and is stored as plain text — and I can explain how I'd harden it."

The other thing I'm proud of is the accuracy work. The first version measured time as 'editor was open' and lines as the net change in line count, which is zero for any refactor. I rewrote the trackers to count gross edits and 'churn', and to only count focused time. I also audited my own code into an IMPROVEMENT_PLAN file with about 30 findings and fixed the serious security ones, with a test that fails if any protected route loses its auth check."

3. The big picture (architecture)



What kind of architecture is this? Say: *"Client-server, REST-ish, a modular monolith."*

- **Client-server:** thin clients (extension, website), one server that owns the truth.
- **Monolith:** one Node process. Features are split into separate route files but deploy together. This is fine for one developer and one database.
- **Modular:** each feature (analytics, groups, goals, leaderboard) is its own file, so you could split one out later. The obvious first candidate is the activity-ingest endpoint.
- **No** message queue, **no** Redis cache, **no** WebSockets. Notifications are fetched by the browser every 30 seconds (polling).

Why not microservices? One developer, one database, and all the features share the same data. Splitting them would add network calls and complexity for no benefit yet.

4. The VS Code extension

How it starts and runs

When VS Code finishes starting up, the extension wakes (`onStartupFinished`), creates five small "trackers", and starts a timer. The timer ticks every **30 seconds**: if you've been idle 2+ minutes it pauses (resuming when you type again); if you've been active and enough time has built up (default half a minute) it sends one activity record.

What it records

Each tracker keeps counters, then hands them over ("snapshot and reset"):

Tracker	What it counts
Editor	characters and lines typed/deleted, churn (wrote then deleted within 10 min), undo/redo, saves, file switches, time reading vs writing
Focus	minutes the window was actually in front, and "flow blocks" (a work stretch that ends after 2 min idle)
Git	commits (via VS Code's built-in Git, so it catches GUI commits too), uncommitted file count
Terminal	commands run, which succeeded/failed, build/test runs, repeated failures
Debug	number of debug sessions

What it sends (the payload)

One JSON object per flush: timestamp (set to the **start** of the interval, not "now", so hour-of-day charts aren't skewed), file **name only** (never the full path), file type, project, language, duration in **seconds**, line counts, and the four tracker summaries.

Privacy: it never sends file contents, diffs, commit messages, or folder paths. Terminal commands are cleaned (`token=...` becomes `token=***`) and shortened.

What happens when things go wrong

Situation	What happens now (REAL)
You go idle	It pauses, then resumes on your next keystroke
VS Code closes	It tries one last send, then stops
No internet	The unsent minutes stay in memory and retry next tick. If VS Code restarts first, that time is lost — there's no on-disk queue
Key is wrong / was reset	You get a one-time popup: "Set API Key" / "Open Dashboard"
Same data sent twice	It gets saved twice and counted twice — no duplicate protection

Where the key is stored on your machine: in VS Code's settings file, as **plain text**. **BETTER:** use VS Code's SecretStorage (the OS keychain).

5. The API key (this is the #1 interview topic)

How it works (REAL)

1. You sign in with Google for the first time.
2. The server creates your user and runs `crypto.randomBytes(32)` → a 64-character key.
3. It's saved on your user record as **plain text**.
4. The website shows it to you. You paste it into the extension.
5. The extension sends it on every request as a header: `x-api-key: <key>`.
6. The server does `User.findOne({ apiKey })` to find you, then saves the activity under your ID.
7. If you think it leaked, you click "Regenerate" — a new key is made, the old one dies.

"Is it just an identifier?" — say NO

An **identifier** (like a username) is safe to show publicly. This key is different — holding it lets you **write data**. So it's a **credential**, not an identifier. More precisely:

- It **identifies** you (one key = one user).
- It **authenticates** you (no second check — having it is enough).
- It's a **bearer token** (whoever holds it can use it).
- It **never expires** and has **no limits** on what it can do.
- It's stored and compared as **plain text**.

Say this line: *"It's basically a password that never changes, stored in plain text. That's an OK choice for a first version — it's one database lookup and works offline — but I'd call it a credential in a review, and I know how to make it safer."*

What an attacker could do

With a stolen key they can **fake activity** for you — send fake hours in a loop and mess up the leaderboard and your charts. They **cannot** read your dashboard (that needs your login cookie). There's no rate limit and no sanity check on the numbers, so it's easy.

How to fix it (BETTER)

- **Store a hash, not the key.** Give the key as `ct_{id}_{secret}`, look up by the `id`, compare the `secret` against a hash. A database leak then reveals nothing usable.
- **Allow several keys per user**, each named ("laptop", "desktop") and revocable on its own.
- **Rotation with a grace period** — the old key still works for 24 hours after you make a new one, so tracking doesn't suddenly break.
- **Even better:** the website hands out a short-lived signed token the extension refreshes, or use an OAuth "device flow" so the extension never holds a long-term secret.

6. The database (MongoDB)

The collections

Collection	What it holds	Notes
activities	one record per flush from the extension	the big one; grows forever
users	Google ID, email, name, the plain-text API key	
groups + groupmembers	study groups + who's in them	groupmembers is a proper join table with a "one row per (group, user)" rule
goals	title, target hours, tech stack, deadline, status	nothing in the code ever marks a goal "completed" — only the demo seed script does
teams	older idea; members stored inside the team record	backend exists but the website never shows it — dead code
notifications	goal deadline reminders	created by the cron job

The activities record

Key fields: `userId` (stored as **text**, not a real reference — a quirk), `file/language/ project`, `duration in seconds` (every reader divides by 3600), `line counts`, `timestamp`, and four sub-objects: `terminalAnalytics`, `editorAnalytics`, `focusAnalytics`, `gitAnalytics`.

Why MongoDB (a good answer)

- Activity records are **write once, never change** — perfect for a document store.
- The shape **grew over time** — I added three tracker sub-objects with **no migration**, because Mongoose just fills missing fields with zero.
- The main read is "give me one user's records for the last week, then add them up" — no joins needed on the hot path.
- Free managed hosting (Atlas).

When SQL (Postgres) would be better

- The **relational bits** — group membership, goal ownership — where foreign keys and "delete cascades" would remove defensive code.
- The **leaderboard** — that's a `GROUP BY user` which SQL does with an index, instead of loading the whole table into Node.
- **Transactions** for multi-step actions.

Indexes

There are indexes on `{userId, date}`, `{userId, project}`, `{userId, language}`. **The gap:** every real query filters by `timestamp`, but the index is on `date`. So those queries scan all of a user's records and filter in memory. **BETTER:** add `{userId: 1, timestamp: -1}`.

7. The backend and API

How one request flows

Extension sending data: `POST /api/extension/track` → check the API key → basic checks (file, language, duration present) → clean the numbers (missing → 0) → `Activity.create(...)` → return 201.

Website reading data: `GET /api/analytics/:userId` → check the JWT cookie → **check "is this your own data?"** (403 if not) → `Activity.find(last 7 days)` → add it up in JavaScript → return JSON → Chart.js draws it.

The main endpoints

What	Endpoint	Login	Notes
Send activity	<code>POST /api/extension/track</code>	API key	one record per call
Verify key	<code>GET /api/extension/verify</code>	API key	used during setup
Daily / weekly charts	<code>GET /api/analytics/:userId /weekly/:userId</code>	JWT + ownership check	adds up in JavaScript
Insights	<code>GET /api/metrics</code>	JWT	no :userId in the URL — uses your session only, so it can't leak someone else's data
Leaderboard	<code>GET /api/leaderboard</code>	JWT	reads the whole <code>activities</code> table every time
Groups	<code>/api/groups/*</code>	JWT (+ membership check for details)	
Goals	<code>/api/goals/*</code>	JWT (+ owner check)	
Notifications	<code>/api/notifications/*</code>	JWT	correctly scoped to you
Google login	<code>/auth/google , /auth/google/callback</code>	—	sets the JWT cookie

Weak spots in the API

- Every route does its own `try/catch` and returns `error.message` to the client — that **leaks internal details**. **BETTER:** one central error handler.
- **No rate limiting anywhere** (`express-rate-limit` is installed but not used).
- **No security headers** (`helmet` is installed but not used).
- **No real input validation** — `duration: 999999999` is accepted; you can even send your own `timestamp`.
- No pagination on the leaderboard.

8. The frontend (React)

- **React 19 + Vite + TypeScript + Tailwind.** `react-router-dom` for pages.
- `App.tsx` checks `GET /api/user/profile` on load. If that works, show the app. If not, show the login page.
- **State:** just `useState` in each page, plus passing the `user` object down as a prop. The only React Context is the **theme** (28 colour themes, saved in `localStorage`).
- **No data-fetching library.** `@tanstack/react-query` is installed but **never used**, so every page reload re-fetches everything. **BETTER:** actually use React Query.
- **Charts:** `Chart.js` (Line, Pie, Bar).

Things to know (and own honestly)

- `npm run build` **currently fails** — the TypeScript check reports 29 old errors (mostly unused imports). `vite build` alone works. It's a cleanup job, not a design flaw.
- The dashboard's "**Repeated Failures**" panel shows **fake hard-coded data**, not the real numbers (which the backend does calculate).
- The **Goals page to-do list isn't saved** — it only lives in React state until you refresh.
- The **Teams page isn't linked** anywhere.

9. Insights — "is this machine learning?"

No. Say it plainly. It's plain statistics, calculated fresh every time you open the page.

The five metrics (simple definitions)

Metric	What it means	How it's worked out
Deep work ratio	how much focused time was in long stretches	minutes in blocks ≥ 25 min $\div$ total focused minutes
Flow blocks	the shape of your sessions	median, longest, and count of your work stretches
Consistency	how steady you are day to day	$1 - (\text{how spread out your daily minutes are})$ — steady beats bursty
True peak hour	your <i>most productive</i> hour, not your busiest	the hour with the best score of $\text{commits} + \text{lines} - \text{churn}$
Estimation accuracy	do you underestimate goals?	average of (hours you actually did $\div$ hours you guessed)

Key points for the interview

- **Runs on every page load. Not cached. BETTER:** cache it per (user, time window).
- **Needs recent data.** If you're on an old extension version, the focus metrics show "—" instead of "0%".
- **Why statistics instead of ML?** No labelled outcomes to learn from, not enough users, and every number needs to be explainable. Statistics work from day one.
- **The AI part is designed but not built.** There's a plan for an LLM that would *describe* these numbers in sentences — with a strict rule that it can't invent any number — but no code for it yet.

If asked "how would you add real ML?"

Build feature vectors per session (focused minutes, churn ratio, commits, hour of day), normalise per user, then start with **unsupervised clustering** (k-means) to find session types like "deep focus" vs "firefighting". Only do supervised learning once you actually store outcomes like "goal missed". Serve it from a background job, never inside the request.

10. The social features — leaderboard and groups

The leaderboard — how ranking works (**REAL**)

Rank = **total coding hours, all time, highest first**. Ties keep their existing order.

How it's calculated

Every time someone opens the page:

1. Load **every user**.
2. Run an aggregation over the **entire activities collection** — sum hours and lines per user.
3. Merge and sort **in Node**.
4. Give each user scores (speed, quality, etc.) **relative to the current top user** — so everyone's score shifts when the leader codes more.

The problems

- **It reads the whole activity table on every request.** No time window, no cache, no pagination. This is the **first thing that breaks** as data grows.
- It returns **every user's email**.
- "Commits" on the board is actually **record count**, not real git commits (mislabelled).

The fix (**BETTER**) — know this well

1. Keep a small **UserStats table**: one row per user with running totals, updated with `$inc` whenever activity comes in.
2. The leaderboard becomes `UserStats.find().sort().limit(50)` — reads 50 rows, not millions.
3. For "what's my rank?" instantly, use a **Redis sorted set**: `ZADD` on update, `ZREVRANGE` for a page, `ZREVRANK` for your position — all $O(\log n)$.
4. Add time-windowed tables for "this week" boards. Paginate. Cache the top 50.

Groups

- **Create**: name, description, public/private. Private groups get a **hashed** (scrypt) password. The creator becomes the first member.
- **Join**: public = one click; private = enter the password (old plain-text ones still work and get upgraded to a hash on the next correct login).
- **Membership** is a `groupmembers` join table with a "one row per (group, user)" unique rule — the one solid race protection in the whole app.
- **Leave**: the last member leaving deletes the group. **View details**: members-only (403 otherwise), then a group leaderboard (same slow full-scan as the global one).

Group weak spots: no admin/owner powers (`createdBy` is stored but unused — no kick or rename); a double-join returns **500** instead of **409**; "discover" lists private groups too (password-gated, not hidden); no rate limit on join, so passwords can be guessed.

11. Login and security

The website login flow

1. Click login → go to `/auth/google` → Google → back to `/auth/google/callback`.
2. The server finds or creates your user, then signs a **JWT** with your ID, name, email. Valid for **1 day**.
3. The JWT goes into an **httpOnly cookie** (JavaScript can't read it).
4. Every protected route reads the cookie, verifies the JWT, loads the user.
5. Logout clears the cookie.

Login weak spots

- The signing secret has an unsafe fallback (`'your_jwt_secret'`) if the env var is missing.
- **No refresh token** — after 1 day you're just logged out.
- **No way to revoke** a token early (no server-side session list).
- The cookie is `sameSite: 'none'` in production with no CSRF token — low impact here because most actions are POSTs with a JSON body, but worth mentioning.

Security — already fixed (on the current branch)

- Analytics endpoints now check **"is this your data?"** (was: anyone with your ID could read your history).
- Old **unauthenticated** write/read endpoints **deleted**.
- Group passwords are now **hashed** (were plain text).
- The `AUTH_BYPASS` dev flag is **refused in production**.
- Goal and team endpoints now check ownership/membership.
- There's a test that **statically scans the routes** and fails if any protected route loses its auth middleware.

Security — still open (be honest about these)

Issue	Why it matters
API key is plain text, never expires, no limits	leak = someone fakes your activity
No rate limiting anywhere	login stuffing, activity flooding, password guessing
No security headers (<code>helmet</code> unused)	missing basic browser protections
No input validation on activity	<code>duration: 1e12</code> accepted; fake data is trivial
Errors return <code>error.message</code>	leaks internal details
Leaderboard shows every email	privacy
No duplicate/replay protection	resend a request → counted again
Group search puts user text straight into a regex	slow-regex (ReDoS) risk

"Can users cheat the leaderboard?" — yes, easily. `curl` the track endpoint with `duration: 3600` in a loop using a valid key; one call = one fake hour. Fix: validate the duration (1–3600s), rate-limit per key, require a unique ID per send, and only trust hours that also have editor/focus/git activity.

12. Scaling up — 10k, 100k, 1M users

Users	What happens	What to do
10,000	Mostly fine. The leaderboard starts getting slow once <code>activities</code> is tens of millions of rows.	Add the missing <code>timestamp</code> index. Put a 30-day window + limit on the leaderboard query.
100,000	Leaderboard is unusable without a rollup. No caching means the same maths runs over and over.	Build the <code>UserStats</code> rollup. Move analytics from JavaScript into MongoDB <code>\$group</code> .
1,000,000	One server + one database is the wrong shape.	Put a queue in front of activity ingest → workers write to a sharded <code>activities</code> (split by <code>userId</code>) plus rollups. Redis for the leaderboard and the insights cache. Serve charts from precomputed rollups. Archive old raw records. Add real monitoring.

The one-line answer: *"It's fine to about 10k users. The first thing to break is the leaderboard, because it reads the whole activity table on every request. The fix is a small per-user stats table updated on write, which turns a full-table scan into a 50-row read."*

13. Testing and deployment

Testing — the honest picture

What exists: small test scripts using Node's built-in `assert` (no framework).

- **Backend:** 6 files, ~57 checks — the streak logic, the number-cleaning functions, the five insight formulas, the ownership helper, the password hashing, and the route scanner that enforces auth.
- **Extension:** 2 files — the trackers' behaviour, and an "activation" test that loads the real built bundle and checks every command is registered and no network call happens without a key.

What's missing:

- **No integration tests** — nothing starts the server and hits a real database.
- **No frontend tests at all.**
- Nothing has ever run against a real database — the queries are only reasoned about.

BETTER: add `supertest` + an in-memory MongoDB for API tests; React Testing Library for the frontend; one end-to-end test with Playwright.

Deployment

- **Website:** Vercel. Build is `tsc -b && vite build` (currently failing at the `tsc` step).
- **Backend:** Render (a normal always-on Node process). There's also a Vercel serverless config in the repo.
- **Database:** MongoDB Atlas.
- **Extension:** packaged with `vsce`, published to the VS Code Marketplace.
- **No CI/CD, no Dockerfile, no automated tests on push.**

One gotcha: the deadline-reminder cron job uses `node-cron`. That works on Render (always on) but **breaks on serverless** — the process is frozen between requests, so the hourly job never fires. **BETTER:** use Vercel Cron or an external scheduler hitting a protected endpoint.

14. Quick Q&A (memorise these)

Why MongoDB? Activity records are write-once and the shape changed over time — I added three sub-objects with no migration. Main read is "one user's week, then sum". No hot-path joins.

Why not Postgres? Honestly, the relational parts (groups, goals) and the leaderboard would be better in SQL. The activity stream suits documents. If I redid it with the social features front of mind, I might pick Postgres.

Why a monolith? One developer, one database, shared data model. I'd split out the activity-ingest endpoint first when volume demands it.

Is the API key authentication? Yes — it's a bearer credential, not an identifier. Plain text, no expiry, no scope.

What if someone steals a key? They can fake your activity and mess up the leaderboard. They can't read your dashboard — that needs the login cookie.

How would you fix the key? Hash it at rest with an ID prefix, allow multiple per user, rotation with a grace period — or move to short-lived tokens / OAuth device flow.

Why not OAuth for the extension? A device flow is the right answer; the key was faster to ship. It's a legitimate MVP trade-off.

Why JWT and not sessions? Stateless — no session store to run, survives restarts. The cost is no easy revocation and no refresh token.

How does data get from the extension to the dashboard? Extension event → tracker counters → 30s flush → check API key → save one record → (later) website asks → check JWT + ownership → read records → add up → JSON → chart.

Where's the aggregation done? Mostly in JavaScript after `Activity.find()`. That's tech debt — it should be MongoDB `$group`. The streak and the insights already use `$group`.

What's the bottleneck? The leaderboard — it reads the whole activity table on every request.

How do you scale the leaderboard to 10M users? A per-user `UserStats` rollup updated on write, then a Redis sorted set for $O(\log n)$ rank lookups.

Is Insights machine learning? No — plain statistics (averages, medians, a spread score), computed fresh each load. An LLM description layer is designed but not built.

Why statistics not ML? No labelled data, not enough users, and every number must be explainable.

Can users cheat the leaderboard? Easily — send fake hours with a valid key. There's no rate limit or validation. Fix: validate duration, rate-limit, require a unique send ID, cross-check with editor activity.

What happens if the same activity is sent twice? It's saved twice. No duplicate protection. Fix: a client-generated ID per send + a unique index.

Two requests at the same time — any problem? Activity insert is a single atomic write, so that's fine. The weak spot is group-join (unique rule catches it but returns a 500) and adding a team member (a read-modify-write that could lose an update — should use `$addToSet`).

Why is `userId` a string on activities but an ID everywhere else? Historical. It forces type-conversion tricks in the leaderboard and blocks joins. Migrating it is on the list.

What breaks first at scale? Leaderboard, then the un-indexed analytics scans, then the serverless cron.

Your frontend doesn't build — explain. 29 old TypeScript errors, mostly unused imports. `vite build` works; the combined `tsc -b && vite build` fails. A cleanup, not a design issue.

How do you know the database queries are correct if nothing ran against a real DB? I don't fully — the pure functions are unit-tested (and the streak test runs the *shipped* code, not a copy), but there's no integration test. That's the biggest gap and it's on the plan.

What was the hardest part? Making the *time* number honest — idle time, and lines that net to zero on a refactor. I built an idle/pause state machine and switched to gross edit counts plus a churn metric.

What would you improve first? Hash the keys, build the leaderboard rollup, move analytics into the database, fix the TypeScript errors, add integration tests.

15. "Defend your project" — the tough follow-ups

"Why MongoDB?" → "But you have a join table and embedded arrays — isn't that SQL fighting a document store?"

Partly true. The join table is me reaching for a relational pattern because groups need it, and it works — the unique index gives the guarantee. It's a sign the social half would be happier in Postgres. The activity half, which is the bulk and the hot path, genuinely suits documents.

"Your leaderboard is $O(n)$ per request." → "A cache fixes that."

A cache hides the cost behind a timer — the full scan still happens. The real fix is the rollup table, which changes the complexity. A cache on top of that is a nice extra.

"Why not Redis / a queue / WebSockets?"

Nothing needs them yet. Leaderboard and insights finish in under a second at demo scale. Redis is the obvious next dependency (leaderboard + cache + rate limits). A queue earns its place past ~10k concurrent users. WebSockets I'd add for real-time group activity, not for a 30-second notification poll.

"You could have shipped faster with Firebase."

True for the CRUD and auth. But the analytics are custom aggregations over a high-volume collection, and Firebase's per-document-read pricing would hurt — the daily chart alone reads a week of records. And the extension needs a stable server API regardless.

"How do you stop cheating if a determined user spreads fake data thinly?"

Cross-check signals. A real coding hour has editor edits and focus events. Weight the leaderboard on corroborated activity, and cap how much any single send can contribute.

"Nothing is tested against a real database — how do you trust it?"

I trust the pure logic (unit-tested, and the streak test runs the real shipped code). I don't fully trust the queries end-to-end — that's stated in my notes. `supertest` plus an in-memory MongoDB closes it and it's the top testing item.

"If this got 10,000 real users tomorrow, what's your first move?"

Add the `timestamp` index and put a 30-day window on the leaderboard query as a stopgap, then build the `UserStats` rollup. In parallel: turn on `helmet` and rate limiting so ingest can't be flooded, and add validation so `duration` can't be faked. Then a health check and real logging.

16. Final reminders

The 10 weaknesses to own before they're pointed out

1. **API key** — plain text, never expires, no limits.
2. **Leaderboard** — reads the whole table every request; no cache, no pagination; leaks emails.
3. **Analytics maths runs in JavaScript**, not the database — downloads every record to add it up.
4. **Missing timestamp index** — queries filter on a field that isn't indexed.
5. **Cron breaks on serverless.**
6. **No rate limiting / security headers / input validation** (libraries installed, not wired up).
7. **Frontend build fails** (29 old TypeScript errors).
8. **Fake data on the dashboard** ("Repeated Failures" panel); **unsaved to-dos**; **dead Teams page**.
9. **Idle time under 2 minutes still counts** as coding time.
10. **No integration tests; nothing tested against a real database.**

For each one: know *why it matters*, *what it would take to fix*, and *why it's acceptable for a student project right now*.

Don't trip on these

- Don't call Insights "AI" or "ML". It's statistics. Say so first.
- Don't say the leaderboard is "optimised". It's a full scan. Know the rollup fix.
- Don't say the API key is "just an identifier". It's a credential.
- Don't claim the frontend builds cleanly. It doesn't (`tsc` fails).
- Don't claim integration test coverage. There is none against a real database.
- Don't claim the extension has an offline queue. It's memory-only.
- The dashboard "Repeated Failures" panel is fake data — don't demo it as real.
- There are 3 contributors — describe what *you* can explain in depth, don't over-claim.

Five sentences that make you sound senior

1. "The API key is basically a plain-text password with no expiry — a fine MVP, but I'd hash it at rest with an ID prefix and add rotation with a grace window."
2. "The leaderboard reads the whole activity table per request; the fix is a per-user stats rollup updated on write, turning a full scan into a 50-row read, with a Redis sorted set for rank lookups."
3. "Insights is plain statistics — a spread score, weighted scoring, medians — chosen so every number is explainable; the LLM layer is designed with a 'can't invent numbers' rule but not built."
4. "Analytics currently add up in JavaScript after loading every record; moving that into MongoDB `$group` is the prerequisite for scale, and the insights code already does it right."
5. "I audited my own code into an improvement plan — about 30 findings by severity — and closed the serious security ones with a test that fails if a protected route loses its auth check."